

Modual-3 Advance Python Programmi

1) Introduction to the print() function in Python.

The print() function in Python is used to display output on the screen. It is a built-in function that helps programmers show messages, values, variables, and results of calculations to the user.

It is one of the most basic and commonly used functions in Python. Beginners use the print() function to understand how programs work and to check program output.

Features of print() Function

- Displays text, numbers, and variable values.
- Can print single or multiple items together.
- Helps in debugging and testing programs.
- Easy to use and beginner-friendly.

Syntax

```
print(object)
```

Example

```
print("Hello Python")
```

Output

Hello Python

The print() function plays an important role in Python programming because it allows communication between the program and the user.

2) Formatting outputs using f-strings and format()

1. f-strings in Python

f-strings (formatted string literals) were introduced in Python 3.6. They allow variables and expressions to be inserted directly inside a string using curly braces {}.

Features of f-strings

- Easy and simple to use
- Improves readability
- Faster than older formatting methods
- Allows expressions inside strings

Syntax

```
f"string {variable}"
```

Example

```
name = "Mira"  
age = 20
```

```
print(f"My name is {name} and I am {age} years old.")
```

Output

My name is Mira and I am 20 years old.

2. format() Method in Python

The format() method is used to insert values into a string using placeholders {}.

Features of format()

- Works in older Python versions
- Useful for arranging multiple values
- Supports positional and named formatting

Syntax

```
"string {}".format(value)
```

Example

```
name = "Mira"  
age = 20
```

```
print("My name is {} and I am {} years old.".format(name, age))
```

Output

My name is Mira and I am 20 years old.

3) Using the input() function to read user input from the keyboard.

Features of input() Function

- Accepts user data from the keyboard
- Makes programs interactive
- Stores input in a variable
- Input is treated as a string by default

Syntax

```
input("message")
```

Example

```
name = input("Enter your name: ")  
print("Welcome", name)
```

Output

```
Enter your name: Mira  
Welcome Mira
```

Taking Number Input

To use numbers, input must be converted using int() or float().

Example

```
age = int(input("Enter your age: "))  
print("Age is:", age)
```

Importance of input()

- Helps create interactive programs
- Used in forms, calculators, games, and applications
- Allows users to provide data during execution

4) Converting user input into different data types (e.g., int, float, etc

Common Data Type Conversion Functions

Function	Purpose
int()	Converts input into an integer
float()	Converts input into a decimal number
str()	Converts data into a string
bool()	Converts data into Boolean value

1. Converting Input to Integer

The int() function converts user input into an integer value.

Example

```
age = int(input("Enter your age: "))  
print(age)
```

Output

```
Enter your age: 20  
20
```

2. Converting Input to Float

The float() function converts input into a decimal number.

Example

```
price = float(input("Enter price: "))  
print(price)
```

Output

```
Enter price: 99.5  
99.5
```

3. Converting Input to String

The `str()` function converts data into string format.

Example

```
num = 100  
text = str(num)
```

```
print(text)
```

4. Converting Input to Boolean

The `bool()` function converts values into True or False.

Example

```
value = bool(1)  
print(value)
```

Output

True

5) Opening files in different modes ('r', 'w', 'a', 'r+', 'w+')

Syntax

```
open("filename", "mode")
```

- filename → Name of the file
- mode → Type of operation to perform

File Opening Modes in Python

1. 'r' Mode (Read Mode)

The 'r' mode is used to read data from a file.

The file must already exist, otherwise Python shows an error.

Example

```
file = open("data.txt", "r")
```

Features

- Default file mode
- Only reading is allowed
- File pointer starts at the beginning

2. 'w' Mode (Write Mode)

The 'w' mode is used to write data into a file.

If the file already exists, old data is erased.

If the file does not exist, a new file is created.

Example

```
file = open("data.txt", "w")
```

Features

- Creates a new file if needed
- Removes previous content
- Used for fresh writing

3. 'a' Mode (Append Mode)

The 'a' mode is used to add new data at the end of an existing file without deleting old data.

Example

```
file = open("data.txt", "a")
```

Features

- Keeps existing data safe
- Adds new content at the end
- Creates file if it does not exist

4. 'r+' Mode (Read and Write Mode)

The 'r+' mode allows both reading and writing in a file. The file must already exist.

Example

```
file = open("data.txt", "r+")
```

Features

- Supports reading and writing
- Does not erase existing data
- File pointer starts from beginning

5. 'w+' Mode (Write and Read Mode)

The 'w+' mode allows both writing and reading. It deletes existing data if the file already exists.

Example

```
file = open("data.txt", "w+")—————
```


6) Using the open() function to create and access files.

Syntax

```
open("filename", "mode")
```

Parameters

- filename → Name of the file
- mode → Type of operation to perform on the file

Creating a File

A file can be created using 'w', 'a', or 'x' mode.

Example

```
file = open("data.txt", "w")  
file.close()
```

Explanation

- Creates a file named data.txt
- If the file already exists, old data is removed in 'w' mode

Accessing a File for Reading

The 'r' mode is used to access and read data from a file.

Example

```
file = open("data.txt", "r")  
print(file.read())  
file.close()
```

Explanation

- Opens the file in read mode
- Displays the file content

Writing Data into a File

The 'w' mode is used to write data into a file.

Example

```
file = open("data.txt", "w")  
file.write("Hello Python")  
file.close()
```

Explanation

- Writes text into the file
- Replaces old content with new data

Appending Data into a File

The 'a' mode adds data at the end of the file.

Example

```
file = open("data.txt", "a")  
file.write(" Welcome")  
file.close()
```

Explanation

- Adds new content without deleting old data

Importance of open() Function

- Helps store data permanently
- Used for reading and writing files
- Supports different file operations
- Important for file management in applications

7) Closing files using close()

In Python, the close() function is used to close an opened file after completing file operations such as reading or writing.

Closing a file is important because it releases system resources and saves changes made to the file.

Syntax

```
file.close()
```

Example

```
file = open("data.txt", "w")  
file.write("Hello Python")  
file.close()
```

Explanation

- The file is opened using open()
- Data is written into the file
- close() closes the file properly

Importance of close() Function

- Saves file data correctly
- Frees memory and system resources
- Prevents data corruption
- Avoids accidental file changes

8) Reading from a file using read(), readline(), readlines().

Python provides different methods to read data from a file. These methods help programmers access file content in different ways.

Before reading a file, it must be opened in read mode using 'r'.

1. read() Method

The read() method is used to read the entire content of a file at once.

Syntax

```
file.read()
```

Example

```
file = open("data.txt", "r")
print(file.read())
file.close()
```

Features

- Reads complete file content
- Returns data as a string
- Useful for small files

2. readline() Method

The readline() method is used to read one line at a time from a file.

Syntax

```
file.readline()
```

Example

```
file = open("data.txt", "r")
print(file.readline())
file.close()
```

Features

- Reads only one line
- Useful for line-by-line reading
- Returns a single string

3. readlines() Method

The readlines() method reads all lines of a file and stores them in a list.

Syntax

```
file.readlines()
```

Example

```
file = open("data.txt", "r")  
print(file.readlines())  
file.close()
```

Features

- Reads all lines together
- Returns output as a list

9) Writing to a file using write() and writelines().

Python provides methods to write data into files. The two commonly used methods are write() and writelines().

To write into a file, the file must be opened in write mode ('w') or append mode ('a').

1. write() Method

The write() method is used to write a single string into a file.

Syntax

```
file.write(data)
```

Example

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Explanation

- Opens the file in write mode
- Writes text into the file
- Creates the file if it does not exist

Features

- Writes one string at a time
- Old data is removed in 'w' mode
- Returns the number of characters written

2. writelines() Method

The writelines() method is used to write multiple lines into a file.

Syntax

```
file.writelines(list)
```

Example

```
file = open("data.txt", "w")
```

```
lines = ["Python\n", "Java\n", "C++\n"]
```

```
file.writelines(lines)
```

```
file.close()
```

Explanation

- Writes all list items into the file
- Each line should contain \n for a new line

Features

- Writes multiple strings together
- Accepts list or sequence of strings
- Faster for multiple lines

10) Introduction to exceptions and how to handle them using try, except, and finally.

An exception is an error that occurs during the execution of a program. When an error occurs, the normal flow of the program stops and Python displays an error message.

Exception handling is used to handle these errors properly so that the program does not crash unexpectedly.

Python provides try, except, and finally blocks for exception handling.

1. try Block

The try block contains code that may produce an error.

Syntax

```
try:  
    # risky code
```

2. except Block

The except block handles the error if an exception occurs in the try block.

Syntax

```
except:  
    # error handling code
```

3. finally Block

The finally block always executes whether an exception occurs or not.

It is commonly used for cleanup operations such as closing files or releasing resources.

Syntax

```
finally:  
    # cleanup code
```

Example of Exception Handling

```
try:  
    num = int(input("Enter a number: "))  
    result = 10 / num  
    print(result)  
  
except ZeroDivisionError:  
    print("Cannot divide by zero")  
  
except ValueError:  
    print("Invalid input")  
  
finally:  
    print("Program ended")
```

Explanation

- try block checks the risky code
- except block handles errors
- finally block executes at the end

11) Understanding multiple exceptions and custom exceptions.

In Python, a program may produce different types of errors during execution. These errors can be handled using multiple except blocks. Python also allows programmers to create their own exceptions called custom exceptions.

1. Multiple Exceptions

Multiple exceptions are used when a program can generate different types of errors. Separate except blocks are written for each error type.

Example

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2
    print(result)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Invalid input")

except Exception:
    print("Some other error occurred")
```

Explanation

- ZeroDivisionError occurs when dividing by zero
- ValueError occurs when invalid data is entered
- Exception handles any other unexpected error

Advantages of Multiple Exceptions

- Handles different errors separately
- Improves program stability

2. Custom Exceptions

Custom exceptions are user-defined exceptions created by programmers for specific situations in a program.

Custom exceptions are created using the raise keyword and by inheriting from the Exception class.

Syntax

```
class MyError(Exception):  
    pass
```

Example

```
class AgeError(Exception):  
    pass  
  
age = int(input("Enter age: "))  
  
try:  
    if age < 18:  
        raise AgeError  
    else:  
        print("Eligible")  
  
except AgeError:  
    print("Age must be 18 or above")
```

Explanation

- AgeError is a custom exception
- raise keyword generates the exception
- except block handles the custom error

12) Understanding the concepts of classes, objects, attributes, and methods in Python.

Python is an object-oriented programming language (OOP). Object-oriented programming is based on the concept of classes and objects. It helps in organizing programs and reusing code efficiently.

1. Class in Python

A class is a blueprint or template used to create objects. It defines the properties and behaviors that objects will have.

Classes contain attributes and methods.

Syntax

```
class ClassName:  
    pass
```

Example

```
class Student:  
    pass
```

2. Object in Python

An object is an instance of a class. It is created from a class and can access the attributes and methods of that class.

Example

```
class Student:  
    pass
```

```
s1 = Student()
```

Explanation

- Student is a class
- s1 is an object of the class

3. Attributes in Python

Attributes are variables that store data related to an object or class.

They represent the properties of an object.

Example

```
class Student:  
    name = "Mira"  
    age = 20
```

Explanation

- name and age are attributes

4. Methods in Python

Methods are functions defined inside a class. They describe the behavior or actions of objects.

Example

```
class Student:  
  
    def show(self):  
        print("Hello Student")
```

Explanation

- show() is a method
- self refers to the current object

13) Difference between local and global variables.

Local Variables

A local variable is a variable declared inside a function. It can only be used within that function.

Features

- Declared inside a function
- Accessible only within the function
- Destroyed after function execution

Example

```
def display():  
    x = 10  
    print(x)
```

display()

Explanation

- x is a local variable
- It cannot be accessed outside the function

Global Variables

A global variable is declared outside all functions and can be accessed anywhere in the program.

Features

- Declared outside functions
- Accessible throughout the program
- Exists during the entire program execution

Example

```
x = 20
```

```
def display():  
    print(x)
```

```
display()  
print(x)
```

Explanation

- x is a global variable
- It can be used both inside and outside the function

Difference Between Local and Global Variables

Local Variable	Global Variable
Declared inside a function	Declared outside functions
Accessible only inside the function	Accessible throughout the program
Temporary variable	Permanent during program execution
Cannot be used outside the function	Can be used inside and outside functions

14) Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python

1. Single Inheritance

In single inheritance, one child class inherits from one parent class.

Example

```
class Parent:
    def show(self):
        print("Parent Class")
```

```
class Child(Parent):
    pass
```

```
c = Child()
c.show()
```

2. Multilevel Inheritance

In multilevel inheritance, a class inherits from another child class.

Example

```
class Grandparent:
    def show1(self):
        print("Grandparent Class")
```

```
class Parent(Grandparent):
    def show2(self):
        print("Parent Class")
```

```
class Child(Parent):
    pass
```

```
c = Child()
c.show1()
c.show2()
```


3. Multiple Inheritance

In multiple inheritance, one child class inherits from more than one parent class.

Example

```
class Father:
    def show1(self):
        print("Father Class")

class Mother:
    def show2(self):
        print("Mother Class")

class Child(Father, Mother):
    pass

c = Child()
c.show1()
c.show2()
```

4. Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from one parent class.

Example

```
class Parent:
    def show(self):
        print("Parent Class")

class Child1(Parent):
    pass

class Child2(Parent):
    pass

c1 = Child1()
c2 = Child2()

c1.show()
c2.show()
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

Example

```
class A:
    def show1(self):
        print("Class A")

class B(A):
    def show2(self):
        print("Class B")

class C(A):
    def show3(self):
        print("Class C")

class D(B, C):
    pass

d = D()
d.show1()
d.show2()
d.show3()
```

Advantages of Inheritance

- Promotes code reusability
- Reduces code duplication
- Improves program organization
- Supports hierarchical classification

15) Using the super() function to access properties of the parent class.

The super() function in Python is used to access the methods, constructors, and properties of a parent class from a child class. It is mainly used in inheritance to reuse the functionality of the parent class.

Instead of directly calling the parent class name, the super() function provides a simpler and more flexible way to access parent class members.

Features of super()

- Accesses parent class methods and variables
- Calls parent class constructors
- Supports code reusability
- Reduces code duplication
- Improves readability and maintainability

Syntax

```
super().method_name()
```

Working of super()

When a child class inherits from a parent class, the super() function allows the child class to use the existing features of the parent class without rewriting the same code.

It helps establish communication between parent and child classes.

Example

```
class Parent:
```

```
    def display(self):  
        print("Parent Class Method")
```

```
class Child(Parent):
```

```
    def show(self):  
        super().display()  
        print("Child Class Method")
```

Advantages of super()

- Reuses parent class code efficiently
- Makes inheritance easier to manage
- Useful in single and multiple inheritance
- Helps avoid direct use of parent class names

16) Method overloading: defining multiple methods with the same name but different parameters

Method overloading is a feature in object-oriented programming where multiple methods have the same name but different parameters. The methods perform different tasks based on the number or type of arguments passed.

Python does not support method overloading directly like some other programming languages. However, it can be achieved using default arguments or variable-length arguments.

Features of Method Overloading

- Same method name is used
- Methods differ in parameters
- Improves code readability
- Increases flexibility of programs

Example Using Default Arguments

```
class Demo:
```

```
    def add(self, a=None, b=None):
```

```
        if a != None and b != None:
            print(a + b)
```

```
        else:
            print(a)
```

```
d = Demo()
```

```
d.add(10)
```

```
d.add(10, 20)
```

Explanation

- The same add() method is used with different parameters
- d.add(10) works with one argument
- d.add(10, 20) works with two arguments

17) Method overriding: redefining a parent class method in the child class.

Method overriding is a feature of object-oriented programming in which a child class provides a new implementation of a method that is already defined in the parent class.

In method overriding, the child class method has the same name and parameters as the parent class method.

It is mainly used to change or extend the behavior of the parent class method.

Features of Method Overriding

- Occurs in inheritance
- Child class redefines parent class method
- Method name remains the same
- Provides specific implementation in child class

Example

```
class Parent:
```

```
    def show(self):  
        print("This is Parent Class")
```

```
class Child(Parent):
```

```
    def show(self):  
        print("This is Child Class")
```

```
c = Child()  
c.show()
```

Output

This is Child Class

SQLite3 and PyMySQL (Database Connectors)

18) Introduction to SQLite3 and PyMySQL for database connectivity.

SQLite3

SQLite3 is a lightweight, file-based database management system built into Python. It does not require a separate server and stores data in a single database file. It is commonly used for small applications, testing, and learning database concepts.

Features:

- Built-in Python module
- No server required
- Easy to use
- Suitable for small projects

PyMySQL

PyMySQL is a Python library used to connect Python programs with a MySQL database server. It allows users to create, read, update, and delete data stored in MySQL databases.

Features:

- Connects Python with MySQL
- Supports SQL queries
- Suitable for large applications
- Requires MySQL server installation

19) Creating and executing SQL queries from Python using these connectors

Python can interact with databases using connectors such as SQLite3 and PyMySQL. These connectors provide methods to execute SQL queries like CREATE, INSERT, UPDATE, DELETE, and SELECT.

Steps:

1. Import the database connector.
2. Establish a database connection.
3. Create a cursor object.
4. Execute SQL queries using the execute() method.
5. Commit changes (for INSERT, UPDATE, DELETE).
6. Fetch data using fetchone() or fetchall().
7. Close the connection.

Example SQL Queries:

- CREATE TABLE – Creates a new table.
- INSERT INTO – Adds records.
- SELECT – Retrieves records.
- UPDATE – Modifies existing records.
- DELETE – Removes records.

Search and Match Functions

20) Using re.search() and re.match() functions in Python's re module for pattern matching.

The re module in Python is used for regular expressions (regex), which help in searching and matching patterns in strings.

re.match()

- Checks for a match only at the beginning of the string.
- Returns a match object if the pattern is found; otherwise, returns None.

Example:

```
import re
```

```
result = re.match("Hello", "Hello World")  
print(result)
```

re.search()

- Searches the entire string for the first occurrence of the pattern.
- Returns a match object if found; otherwise, returns None.

Example:

```
import re
```

```
result = re.search("World", "Hello World")  
print(result)
```

Difference

re.match()	re.search()
Matches only at the beginning of the string	Searches the entire string
Returns match if pattern starts the string	Returns match if pattern appears anywhere

21) Difference Between Search and Match

Search	Match
Searches the entire string for a pattern.	Checks for a pattern only at the beginning of the string.
Pattern can occur anywhere in the text.	Pattern must start from the first character.
More flexible for finding data.	More strict in matching.
Useful when the location of the pattern is unknown.	Useful when validating the start of a string.
Returns a result if the pattern is found anywhere.	Returns a result only if the pattern is found at the start.

